

MIP_ENGINEERING_PLAYBOOK.md

Mainframe Intelligence Platform

Engineering Playbook

Version: 1.0

Purpose

This document defines how engineering teams build, maintain, and extend the Mainframe Intelligence Platform (MIP).

It provides:

- Development standards
 - Copilot usage standards
 - Repository structure
 - Skills architecture
 - Agent architecture
 - Development workflow
 - Coding standards
 - Delivery roadmap
-

Core Engineering Philosophy

Rule #1

Do not build parsers.

Build enterprise knowledge.

Parsers are implementation details.

Knowledge is the product.

Rule #2

Facts before AI.

Always build:

Source ↓ Metadata ↓ Graph ↓ Reasoning ↓ AI

Never:

Source ↓ LLM ↓ Answer

Rule #3

Architecture before implementation.

Never ask Copilot:

"Write the code."

Always ask:

1. Analyze
 2. Design
 3. Review
 4. Improve
 5. Implement
 6. Test
 7. Refactor
-

Workspace Setup

Recommended structure:

mainframe-discovery/

|—— mfcode/ | |—— discovery/ | |—— metadata/ | |—— graph/ | |—— reasoning/ | |——
copilot/ | |—— docs/ | |—— tests/ | |—— .skills/

Local Development Stack

Language:

Python 3.13

Framework:

FastAPI

Validation:

Pydantic v2

Metadata Store:

PostgreSQL

Graph Store:

Neo4j

Caching:

Redis

Observability:

OpenTelemetry

Testing:

pytest

Package Management:

uv

Containerization:

Docker

VS Code Configuration

Required Extensions:

- GitHub Copilot
- GitHub Copilot Chat
- Python
- Ruff
- YAML
- Markdown

Optional:

- Neo4j Extension
 - REST Client
-

Copilot Operating Model

Copilot is treated as:

- Architect
- Engineer
- Reviewer
- Tester
- Documentation Writer

Do not use Copilot as:

- Code Generator Only
-

Global Copilot Instructions

Create:

`.github/copilot-instructions.md`

Contents:

You are a Principal Mainframe Modernization Architect.

Expertise:

- COBOL
- JCL
- DB2
- VSAM
- IMS
- CICS
- T-SYS
- Knowledge Graphs
- Python
- FastAPI
- Neo4j

Rules:

1. Design before implementation.
2. Generate production-grade code only.
3. Include tests.
4. Include logging.
5. Include error handling.
6. Explain architecture decisions.
7. Design for enterprise scale.
8. Prefer reusable components.

Skills Architecture

Location:

.skills/

Each skill should have:

skill.md

Example:

.skills/cobol-analysis/skill.md

Skill Structure

Every skill contains:

Purpose

Responsibilities

Inputs

Outputs

Constraints

Prompt Templates

Success Criteria

Future Extensions

Skill 1

Repository Discovery

Purpose:

Understand repository structure.

Responsibilities:

- Inventory
- Classification
- Metadata generation

Output:

Repository Catalog

Skill 2

COBOL Analysis

Purpose:

Extract metadata.

Responsibilities:

- PROGRAM-ID
- CALL
- COPY
- EXEC SQL

Output:

Program Catalog

Skill 3

JCL Analysis

Purpose:

Discover batch flows.

Output:

Job Catalog

Skill 4

DB2 Analysis

Purpose:

Build lineage.

Output:

Data Catalog

Skill 5

Graph Builder

Purpose:

Build dependency graph.

Output:

Knowledge Graph

Skill 6

Impact Analysis

Purpose:

Determine downstream impacts.

Output:

Impact Reports

Skill 7

Business Rule Discovery

Purpose:

Extract business logic.

Output:

Business Rules Repository

Skill 8

Modernization Advisor

Purpose:

Generate modernization insights.

Output:

Transformation Recommendations

Standard Copilot Workflow

Every task follows this sequence.

Step 1

Architecture Prompt

Example:

Design a scalable COBOL parser.

Requirements:

- Python 3.13
- Pydantic v2
- Enterprise scale

Output architecture only.

Step 2

Architecture Review Prompt

Review the proposed architecture.

Identify:

- Weaknesses
- Risks
- Scalability concerns

Provide recommendations.

Step 3

Implementation Prompt

Generate production-grade implementation.

Include:

- Models
- Services
- Tests
- Logging
- Error handling

Step 4

Code Review Prompt

Perform a principal engineer review.

Identify:

- Bugs
- Complexity
- Maintainability issues

Recommend improvements.

Step 5

Refactoring Prompt

Refactor implementation.

Improve:

- Readability
- Testability
- Performance

Metadata First Development

Never implement graph logic before metadata exists.

Order:

Inventory ↓ Metadata ↓ Relationships ↓ Graph ↓ Reasoning ↓ AI

Phase 1 Deliverables

Repository Discovery

Success Criteria:

Answer:

- What files exist?
 - Where are they located?
-

Phase 2 Deliverables

Program Discovery

Success Criteria:

Answer:

- What programs exist?
 - What copybooks exist?
-

Phase 3 Deliverables

Relationship Discovery

Success Criteria:

Answer:

- Who calls whom?
 - What uses what?
-

Phase 4 Deliverables

Lineage Discovery

Success Criteria:

Answer:

- What tables are updated?
 - What datasets are used?
-

Phase 5 Deliverables

Knowledge Graph

Success Criteria:

Answer:

- Which jobs execute this program?
 - Which programs update this table?
-

Phase 6 Deliverables

Reasoning Layer

Success Criteria:

Answer:

- What breaks if changed?
 - What is the root flow?
-

Phase 7 Deliverables

Enterprise Copilot

Success Criteria:

Natural language questions work.

Examples:

Show payment flow.

Explain this module.

Which jobs update this table?

Trace batch execution.

Engineering Standards

Every component must contain:

- Domain Models
 - Services
 - Unit Tests
 - Structured Logging
 - Error Handling
 - Documentation
-

Code Quality Standards

Mandatory:

- Type Hints
- Ruff
- Pytest
- Pydantic Validation

Avoid:

- Global Variables
 - Hardcoded Paths
 - Monolithic Classes
-

Testing Strategy

Unit Tests

Every parser.

Integration Tests

Parser + Metadata Store

Graph Tests

Relationship validation.

End-to-End Tests

Question answering.

GitHub Workflow

Branch Strategy:

feature/ *bugfix*/ hotfix/*

Pull Requests:

Required:

- Tests
 - Review
 - Documentation
-

MVP Definition

MIP MVP is complete when users can ask:

1. Which jobs execute program X?
 2. Which programs call program X?
 3. Which copybooks does program X use?
 4. Which tables does program X update?
 5. Show dependency flow.
 6. Identify root programs.
-

Future Vision

MIP evolves into:

Enterprise Legacy Intelligence Platform

Capabilities:

- Discovery
- Understanding
- Reasoning

- Modernization
- Transformation

The long-term goal is not code analysis.

The long-term goal is enterprise knowledge intelligence.